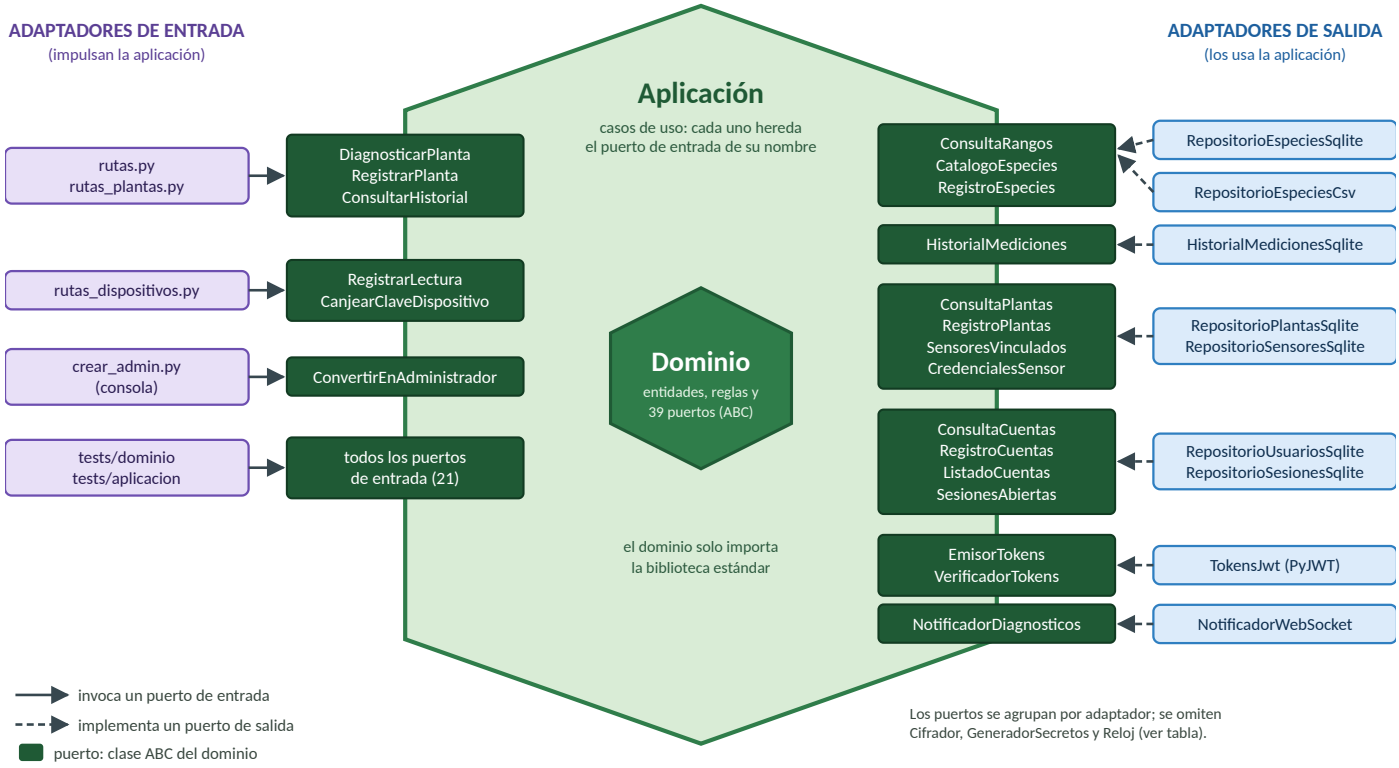
Juan Joya - Roger Bohórquez - Felipe Céspedes

Documento de arquitectura — Matera inteligente

1. Diagrama hexagonal



El diagrama se lee de afuera hacia adentro. Los adaptadores conocen los puertos; el dominio no conoce a ningún adaptador. En modo CSV, `RepositorioEspeciesCsv` reemplaza a `RepositorioEspeciesSqlite` en `ConsultaRangos` y `CatalogoEspecies`.

Las cuatro reglas del diagrama se cumplen y se pueden demostrar:

- `dominio/` solo importa la biblioteca estándar; lo comprueba `tests/dominio/test_arquitectura.py`.
- Las 167 pruebas de dominio y aplicación corren en menos de dos segundos, sin servidor ni base de datos.
- `main.py` es el único punto de composición.
- Ningún adaptador se crea dentro del dominio.

2. Puertos y adaptadores

El dominio define 21 puertos de entrada (`dominio/puertos_entrada.py`) y 18 de salida (`dominio/puertos.py` y `puertos_usuarios.py`), todos como ABC. Cada caso de uso de `aplicacion/` hereda el puerto de entrada de su mismo nombre, y solo `main.py` decide qué adaptador va en cada puerto.

Puerto de entrada (ejemplos)	Adaptador que lo invoca
`DiagnosticarPlanta`, `ListarEspecies`, `RegistrarPlanta`, `ConsultarHistorial`, `ListarPlantas`	`presentacion/rutas.py` y `rutas_plantas.py` (REST del front)
`CanjearClaveDispositivo`, `RegistrarLectura`	`presentacion/rutas_dispositivos.py` (ESP32 y simulador)
`ConvertirEnAdministrador` y todos los demás	`crear_admin.py` (consola); `tests/dominio` y `tests/aplicacion` (dobles)

Puerto de salida	Adaptador	Archivo en `infraestructura/`
`ConsultaRangos`, `CatalogoEspecies`	`RepositorioEspeciesSqlite` o `RepositorioEspeciesCsv` (R4)	`sqlite/repositorio_especies.py`, `repositorio_especies_csv.py`

Puerto de salida	Adaptador	Archivo en `infraestructura/`
`RegistroEspecies`	Con base de datos: `RegistroEspeciesConCopiaCsv`, que envuelve a `RepositorioEspeciesSqlite` y reescribe la copia en CSV. Con CSV: `EspeciesDeSoloLectura`	`repositorio_especies_csv.py`
`HistorialMediciones`	`HistorialMedicionesSqlite`	`sqlite/historial_mediciones.py`
`ConsultaPlantas`, `RegistroPlantas`	`RepositorioPlantasSqlite`	`sqlite/repositorio_plantas.py`
`SensoresVinculados`, `CredencialesSensor`	`RepositorioSensoresSqlite`	`sqlite/repositorio_plantas.py`
`ConsultaCuentas`, `RegistroCuentas`, `ListadoCuentas`	`RepositorioUsuariosSqlite`	`sqlite/repositorio_usuarios.py`
`SesionesAbiertas`	`RepositorioSesionesSqlite`	`sqlite/repositorio_usuarios.py`
`EmisorTokens`, `VerificadorTokens`	`TokensJwt` (PyJWT)	`tokens_jwt.py`
`NotificadorDiagnosticos`	`NotificadorWebSocket`	`tiempo_real.py`
`Cifrador`, `GeneradorSecretos`, `Reloj`	`CifradorPbkdf2`, `GeneradorSecretosSeguro`, `RelojSistema`	`seguridad.py`, `reloj.py`

CSV o base de datos se elige con `MATERA_TABLA_REFERENCIA=csv|bd` (por defecto `bd`). La variable se lee en `infraestructura/configuracion.py`; `main.py:_tabla_de_referencia()` solo la usa para escoger el adaptador, y el dominio no se toca. Los puertos son pequeños para que cada caso de uso pida solo lo que usa: `ConsultarHistorial` lee plantas pero no las crea, así que depende de `ConsultaPlantas` y no de `RegistroPlantas`.

3. JWT: dónde se emite y dónde se verifica

Los tokens se emiten en los casos de uso y se verifican en los adaptadores de entrada. En los dos casos se pasa por un puerto, y el dominio nunca importa PyJWT.

	Token de usuario	Token de dispositivo
Alcance (`alcance`)	usuario	lecturas
Se emite en	`IniciarSesion` (`aplicacion/cuentas.py:84`), por el puerto `EmisorTokens`	`CanjearClaveDispositivo` (`aplicacion/sensores.py:104`), que canjea la clave del sensor, por `EmisorTokens`
Se verifica en	decorador `Autenticacion.con_usuario` (`presentacion/autenticacion.py`), por `VerificadorTokens.verificar_usuario`	decorador `_con_dispositivo` (`presentacion/rutas_dispositivos.py:51`), por `VerificadorTokens.verificar_dispositivo`
El caso de uso recibe	`IdentidadUsuario`	`IdentidadDispositivo`
Duración	7 días	24 horas

Los dos puertos los implementa `TokensJwt` (`infraestructura/tokens_jwt.py`); un token con el alcance equivocado se rechaza al verificar. Los tokens son revocables: llevan un `jti` que el servidor guarda, así que cerrar sesión, desactivar la cuenta o cambiar el sensor los deja sin efecto. El WebSocket también verifica el token, en su primer mensaje (`presentacion/tiempo_real.py`), para que no viaje en la URL.

La regla de propiedad sí es del dominio: `Planta.exigir_dueno()` (`dominio/plantas.py:33`); una planta ajena responde lo mismo que una inexistente. Verificar un token depende de tecnologías concretas (PyJWT y la cabecera `Authorization`): si mañana se usa OAuth, cambia el adaptador y nada más. En cambio, «un usuario solo ve sus plantas» es cierto sin importar cómo llegó la orden (HTTP, MQTT o consola), y por eso vive en el dominio.

4. SOLID en el código

Se justifican cuatro principios, los que tienen el ejemplo más claro en el código.

Principio	Ejemplo en el código	Qué pasaría sin él
D: inversión de dependencias	`TokensJwt` implementa `EmisorTokens` y `VerificadorTokens`, puertos que declara el dominio. `IniciarSesion` solo conoce el puerto, y `main.py` es el único sitio que crea `TokensJwt`.	Si `IniciarSesion` importara PyJWT, cambiar de librería o pasar a OAuth obligaría a tocar el caso de uso, y sus pruebas necesitarían tokens reales.
O: abierto / cerrado	Agregar `rutas_dispositivos.py` y `simulador_esp32.py` no tocó `dominio/`. Cambiar CSV por base de datos es elegir otro adaptador en `_tabla_de_referencia()`.	Cada canal nuevo (ESP32, MQTT, app móvil) o cambio de almacenamiento obligaría a modificar y volver a probar el dominio.
I: segregación de interfaces	`RepositorioPlantasSqlite` implementa dos puertos, `ConsultaPlantas` y `RegistroPlantas`; `main.py` lo pasa dos veces a `RegistrarPlanta(plantas, plantas, …)`. `ConsultarHistorial` solo pide `ConsultaPlantas` y `HistorialMediciones`.	Con un repositorio único, cada doble de prueba implementaría métodos que el caso de uso nunca llama, y las lecturas quedarían atadas a las escrituras.
S: responsabilidad única	`_con_dispositivo` solo hace autenticación: saca el token de la cabecera, lo verifica y comprueba que el código de la URL sea el del token (si no coinciden, responde 403). La regla de negocio la decide `RegistrarLectura`.	Si el decorador validara reglas de negocio, un futuro adaptador MQTT tendría que repetirlas y las dos copias podrían divergir.

5. Prueba de cambio

El adaptador de dispositivos y el simulador entraron sin modificar ningún archivo de `backend/dominio/`. Commit de la prueba: **`d41ca34`**
`https://github.com/Pipelol0723/matera-inteligente/commit/d41ca344f175d6530a705786c224e69777c5615c`

- **Nuevos:** `presentacion/rutas_dispositivos.py`, `presentacion/esquemas_dispositivos.py`, `simulador/simulador_esp32.py` y sus pruebas.
- **Cambiados:** `main.py` para conectarlo, y ajustes menores en `autenticacion.py`, `servidor_web.py` y `historial_mediciones.py`.
- **Retirados:** `rutas_sensores.py` y `herramientas/enviar_medicion.py`, que eran la versión anterior.

El dominio ya hablaba en términos de puertos: `RegistrarLectura` recibe una `IdentidadDispositivo` y un `ReporteDeDispositivo`, sin saber de dónde vienen. Esas piezas se prepararon antes, en el commit `13a3c1b`. El adaptador nuevo solo traduce `POST /api/v1/dispositivos/<codigo>/lecturas` a esa llamada, y `main.py` lo conecta. Lo que no cambió es lo que vive en el dominio: la validación, el historial y la regla de propiedad. Después de la prueba, la revisión final sí tocó el dominio en `ed639b3` (`Usuario.exigir_activo()`) y en `75e4421` (la firma de `HistorialMediciones.agregar`).

6. Si las lecturas llegaran por MQTT

Cambia el adaptador de entrada de los dispositivos y su conexión en `main.py`. Los puertos `RegistrarLectura` y `VerificadorTokens` siguen iguales.

Pieza	¿Cambia?
`dominio/`, `aplicacion/`, y los adaptadores de salida (`sqlite/`, `tokens_jwt.py`, CSV)	No
Adaptador nuevo: suscriptor MQTT, en lugar de `rutas_dispositivos.py` o además de él	Sí, nuevo
`main.py`: conectar el suscriptor a `RegistrarLectura`	Sí
`simulador/simulador_esp32.py` y `firmware/matera-sensor.ino`: publicar en vez de hacer POST	Sí
`requirements.txt`: cliente MQTT	Sí

Como referencia, `main.py` ya marca el caso inverso: el aviso en vivo saldría por MQTT cambiando solo la línea de `NotificadorWebSocket`. La seguridad funcionaría así:

- **El token:** el JWT del dispositivo viajaría como contraseña en el CONNECT al broker o dentro de cada mensaje.
- **La verificación:** el suscriptor lo verificaría con el mismo `VerificadorTokens.verificar_dispositivo`, igual que hoy `_con_dispositivo`.
- **El código del sensor:** iría en el tópico (por ejemplo, `matera/<codigo>/lecturas`), como hoy va en la URL.